

Ôn tập toán học: Đường thẳng và Siêu phẳng

Mục tiêu: Xây dựng bộ phân loại tuyến tính (Linear Classifier) từ con số 0 để hiểu cơ chế lỗi của học sâu.

- Phương trình dự đoán tổng quát (Hàm tuyến tính affine):

$$y_pred = W \cdot x + b \quad (1)$$

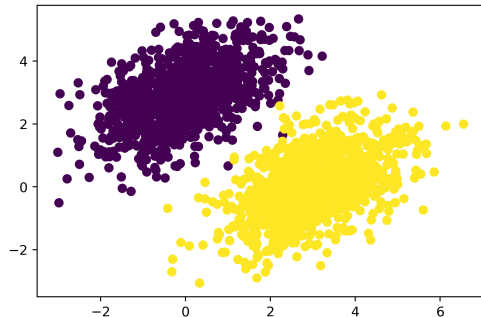
- W (Trọng số - Weights): Quyết định độ nghiêng/hướng của đường thẳng.
- b (Độ lệch - Bias): Quyết định vị trí tịnh tiến của đường thẳng so với gốc toạ độ.

Khởi tạo dữ liệu giả lập (Point Clouds)

Tạo 2 cụm điểm phân bố ngẫu nhiên trong không gian 2D:

- **Cụm 1:** Nhãn (label) = 0.
- **Cụm 2:** Nhãn (label) = 1.

Bài toán: Vẽ một đường thẳng tách biệt chính xác 2 cụm điểm này bằng Gradient Descent.

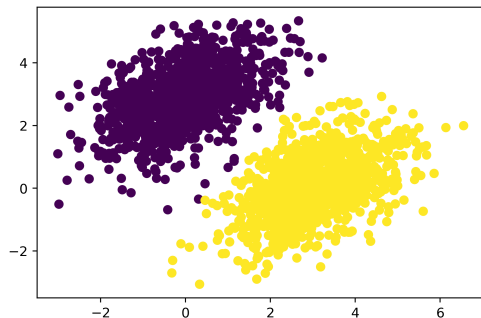


Hình 3.2: Hai cụm điểm dữ liệu ngẫu nhiên

Dự đoán ban đầu (Khởi tạo ngẫu nhiên)

Khi mô hình phân loại vừa được tạo ra:

- W, b ngẫu nhiên rất nhỏ (gần 0).
- Đường thẳng vô nghĩa, điểm MSE Loss rất cao.



Hình 3.3: Đường thẳng phân chia trước khi huấn luyện

Hàm mất mát (MSE) và Vòng lặp huấn luyện

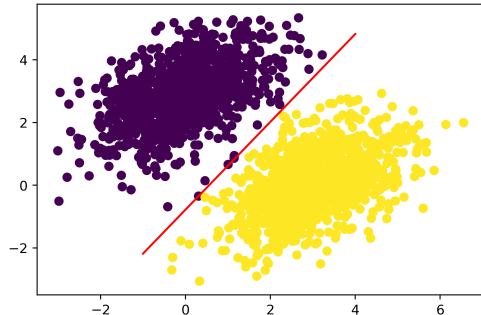
Chúng ta cập nhật W và b dựa trên đạo hàm:

```
1 learning_rate = 0.1
2 for step in range(40):
3     with tf.GradientTape() as tape:
4         predictions = tf.matmul(inputs, W) + b
5         loss = tf.reduce_mean(tf.square(targets - predictions))
6
7     grad_W, grad_b = tape.gradient(loss, [W, b])
8
9     W.assign_sub(grad_W * learning_rate)
10    b.assign_sub(grad_b * learning_rate)
11
```

Kết quả sau Vòng lặp huấn luyện

Sau nhiều bước cập nhật trọng số, bộ phân loại tuyến tính đã tự động xoay và tịnh tiến đường thẳng cắt qua khoảng trống tối ưu.

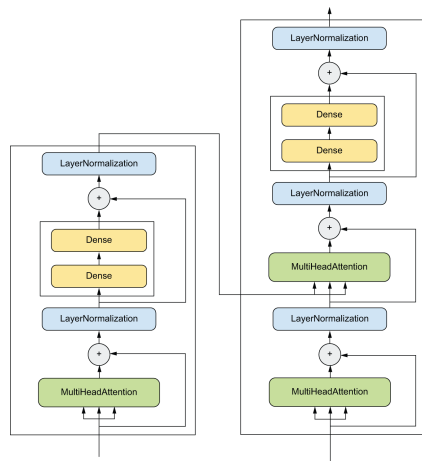
- Thuật toán Gradient Descent hội tụ!



Hình 3.4: Đường thẳng phân chia sau khi huấn luyện

Ứng dụng mở rộng: Kiến trúc phức tạp

- Mạng nơ-ron thực tế kết hợp hàng nghìn phép toán $Wx + b$ đan xen với hàm phi tuyến (ReLU, Softmax).
- Các kiến trúc khổng lồ như **Transformer** (mô hình ngôn ngữ) thực chất vẫn sử dụng nguyên lý Backpropagation và Gradient Descent này để học!



Hình 3.5: Đồ thị kiến trúc của Transformer Block

- **TensorFlow, PyTorch, JAX:** 3 "Cột trụ" Framework cung cấp sức mạnh tính toán tự động đạo hàm cực lớn trên GPU/TPU.
- **Keras:** API thượng tầng (High-level) kết nối và chuẩn hóa toàn bộ sự phức tạp của 3 Framework kia thành các khối block (Layers/Models) gọn gàng.
- **Core Logic:** Cho dù 1 lớp Dense hay 1 mô hình Transformer tỷ tham số, bản chất vẫn là Tính Loss $\rightarrow$ Lấy Gradient $\rightarrow$ Cập nhật W, b .